

Homework 2: Setup and Intro to ROS 2

Overview and Deliverables

1. Reading & Reflection
 - 1:** A short (~300 word) reflection
2. GitHub and VS Code Setup
 - 2.4.3:** A **URL** to your private GitHub repository for this class
 - 2.4.4:** An **invite** to my GitHub account (akubota95) to collaborate on your repository
3. Docker Setup and Intro to ROS 2
 - 3.8.2:** A **diagram** of your *ros2_ws*
 - 3.8.3:** A **diagram** demonstrating how messages are sent between your nodes
 - 3.9.4:** A **video** of your nodes running
 - 3.9.6:** Your updated GitHub repository

PART 1: Reading & Reflection

When beginning to explore a new topic, a “survey” or “review” paper is generally a good place to start to get introduced to the major concepts, recent prior work, and open challenges in that space. Then depending on your own project and goals, you can find and read individual work in more detail.

Read the following:

1. Mavrogiannis, C., Baldini, F., Wang, A., Zhao, D., Trautman, P., Steinfeld, D., & Oh, J. “Core challenges of social robot navigation: A survey.” ACM Transactions on Human-Robot Interaction. 2023.
 - a. Sections 1, 3, 5 (2 and 4 optional)

Write a short reflection (~300 words) using the following prompts as guidance. Note that you may also address other topics such as something you found surprising, connections you made to the real world or media, etc.

1. How do concepts we have discussed in lecture so far (physical design, perception) relate to the challenges discussed in Section 3?

PART 2: GitHub and VS Code Setup

1.1 Create a GitHub account

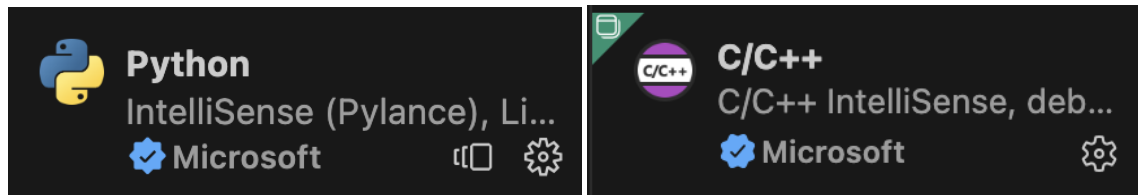
We will be using GitHub in this class. GitHub offers numerous benefits including version control, cloud backup, and collaboration with teammates.

1. If you do not have a GitHub account, create one on the website: <https://github.com/>.

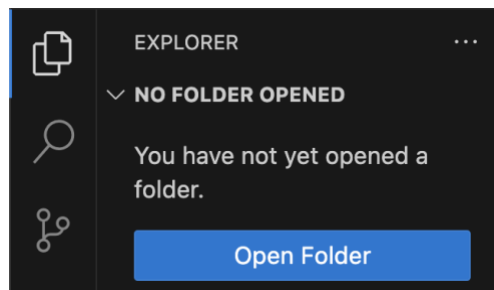
1.2 Set up VS Code

While you can develop entirely using the command line interface (CLI) if you so choose, I recommend programming through the Visual Studio Code (VS Code) integrated development environment (IDE) for quality of life, especially if you are unfamiliar with developing in the terminal.

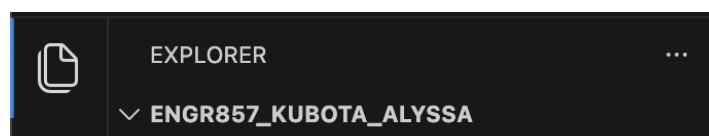
1. Install VS Code for your OS from <https://code.visualstudio.com/download>
2. Install the Python and/or C++ extensions from the Extensions  icon on the left side bar. Search for whichever you plan to use, then select and install the extension.



3. Create a new folder on your machine for this class. Click the Explorer  icon on the left side bar, and then click “Open Folder”.

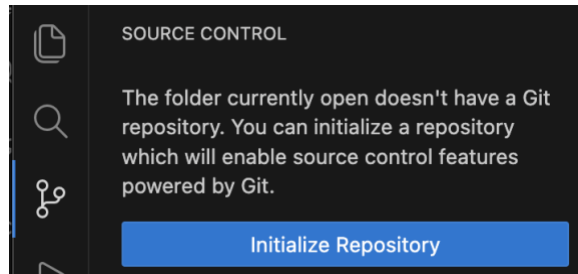


4. Create a new folder named “ENGR857_Lastname_Firstname” and open it.

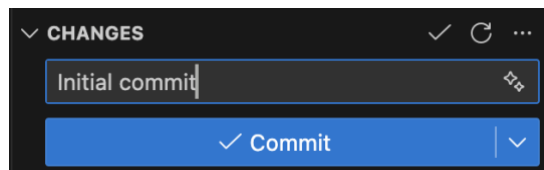


1.3 Connect your directory and push it to GitHub

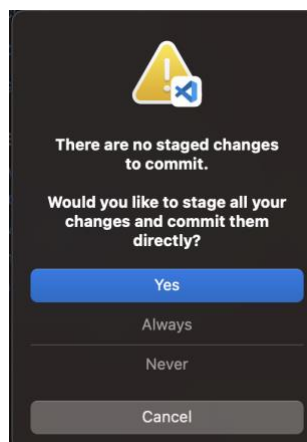
1. Connect your directory to GitHub by clicking on the Source Control  icon on the left side bar, then select “Initialize Repository”. (This is equivalent to running the `git init` command in the command line.)



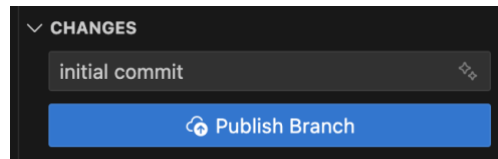
2. If you have not connected VS Code to GitHub before, you may be prompted to do so. A new browser window will open, where you can authorize GitHub for VS Code. Enter your GitHub password to authorize the connection.
3. You will be prompted to enter a commit message. Enter a descriptive message, then click “Commit”. (This is equivalent to running `git commit -m "Initial commit"`)



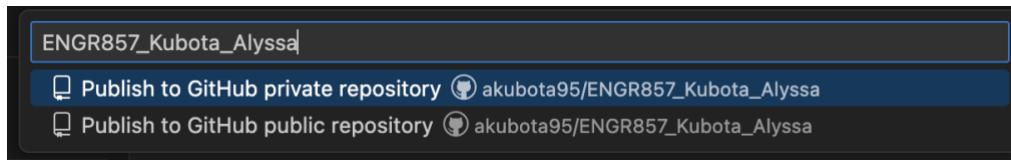
4. If prompted, indicate that “Yes”, you would like to stage your changes to commit.



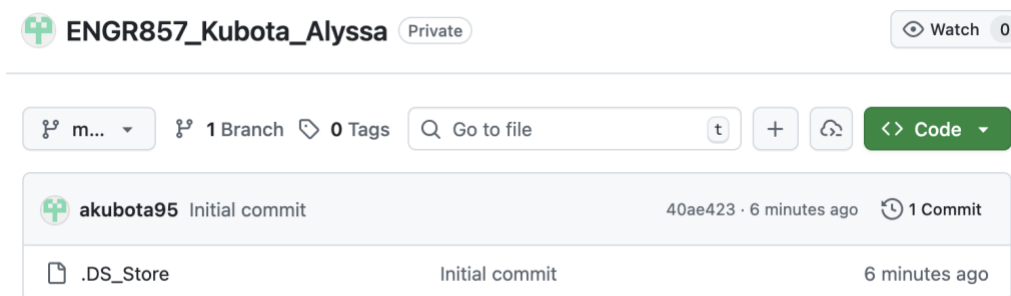
5. Add another message (can be the same as before, then click “Publish Branch”).



6. Choose “Publish to GitHub **private** repository”.



7. You should get a confirmation that your code was successfully committed. Go to the GitHub website that this is true.

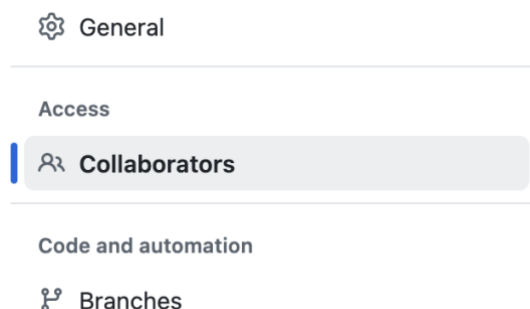


1.4 Add me as a collaborator to your new repository

1. Select Settings from the top menu bar.



2. Select Collaborators on the left side bar



3. Click “Add People” and add me (akubota95).

Manage access



You haven't invited any collaborators yet

Add people

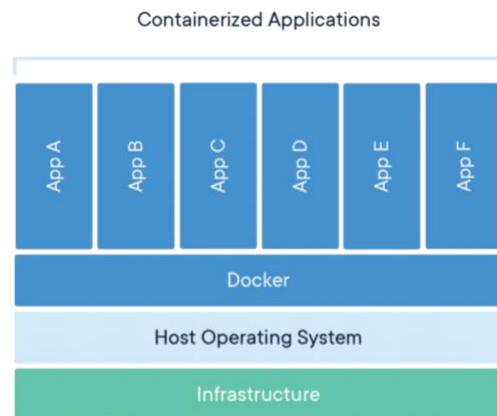
4. **Submit the URL to your repository to Canvas.**

PART 3: Docker Setup and Intro to ROS 2

Adapted from CPE 416 at Cal Poly SLO, [ROS 2 documentation](#), and [Microsoft tutorials](#).

There are many more students taking this course than there are robots to work on! Thus, we will use a Docker container to gain familiarity with ROS 2 on your local machine without needing direct access to the robots.

To give a very brief overview, Docker enables applications to be packaged, distributed, and deployed in standalone *containers*, loosely isolated from their development environment. Containers contain all configurations, libraries, dependencies, and code needed to run the application agnostic of the software installed on the host machine. You can learn more about Docker containers [here](#), or check the [documentation page](#) for more information.



3.1 Install Docker Desktop on your local machine (e.g., personal laptop)

1. Open the Docker Desktop documentation page: <https://docs.docker.com/desktop/>
2. Scroll down and click the hyperlink for your corresponding OS ([Windows](#), [MacOS](#), and [Linux](#) are currently supported).

Next steps

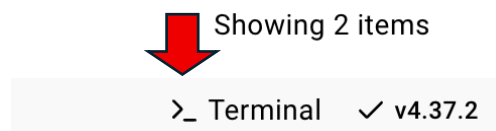
 **Install Docker Desktop**
Install Docker Desktop on [Mac](#), [Windows](#), or [Linux](#).

 **Learn about Docker Desktop**
Navigate Docker Desktop.

3. Follow the installation instructions on the appropriate page.

3.2 Confirm the installation works correctly

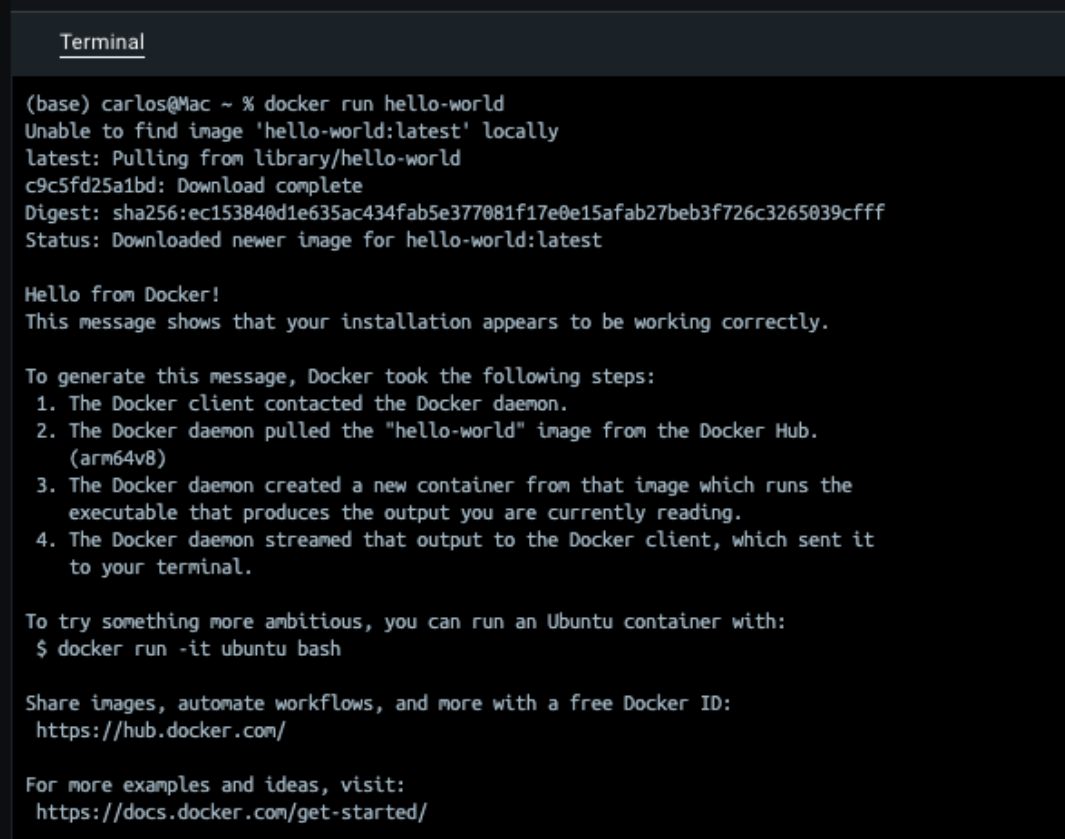
1. Open the Docker Desktop application.
2. Open a terminal within Docker Desktop (button on the bottom right corner).



3. Run the hello-world *image* by running the following command in the terminal:

```
docker run hello-world
```

4. Note that Docker uses the following terminology:
 - a. An *image* (e.g., `hello-world`) contains all the instructions and files needed to run an application (e.g., OS, code, dependencies).
 - b. A *Dockerfile* is used to build an image.
 - c. A *container* is the actual process that executes the application with isolated resources (e.g., CPU, memory). You can run multiple containers from the same image. Containers are designed to be temporary (like a process), while images remain unless deleted (like an application).
5. Read the output and make sure your installation is working correctly. You should see something akin to the following output:

A terminal window titled "Terminal" with a dark background and light text. It shows the command 'docker run hello-world' being executed. The output indicates that the 'hello-world:latest' image was not found locally and was pulled from the Docker Hub library. It shows the digest and status of the download. Below this, a message says 'Hello from Docker!' and 'This message shows that your installation appears to be working correctly.' It then lists the steps Docker took to generate this message: 1. Docker client contacted the daemon, 2. daemon pulled the 'hello-world' image from Docker Hub (arm64v8), 3. daemon created a new container from that image, and 4. daemon streamed the output to the client. It also provides a suggestion to run an Ubuntu container and links to Docker Hub and documentation.

```
Terminal
(base) carlos@Mac ~ % docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
c9c5fd25a1bd: Download complete
Digest: sha256:ec153840d1e635ac434fab5e377081f17e0e15afab27beb3f726c3265039cfff
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (arm64v8)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/
```

This verifies several functionalities are properly configured on your system, including Docker's networking stack (e.g., traffic routing between your host machine and the container) and port mapping (e.g., port forwarding rules are established and functioning).

3.3 Pull a ROS 2 image and build a new image

1. There is a dedicated [Docker repository](#) containing ROS images of various distributions (i.e., a versioned set of ROS packages). The QBots use the [Humble ROS 2 distribution](#).

Pull the docker image for ROS 2 desktop, depending on your processor architecture:

- a. For an x86 image (Intel chips):

```
docker pull osrf/ros:humble-desktop
```

- b. For an arm64 image (Apple Silicone):

```
docker pull arm64v8/ros:humble
```

2. In your *ENGR857* repository, create a file called *Dockerfile* and paste in the following content (note your first line should match the image you pulled earlier).

```
FROM arm64v8/ros:humble

RUN apt-get update && apt-get install -y vim && apt-get install tree
RUN apt install -y python3-pip
RUN apt-get install dos2unix
RUN pip3 install setuptools==58.2.0
RUN echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc
```

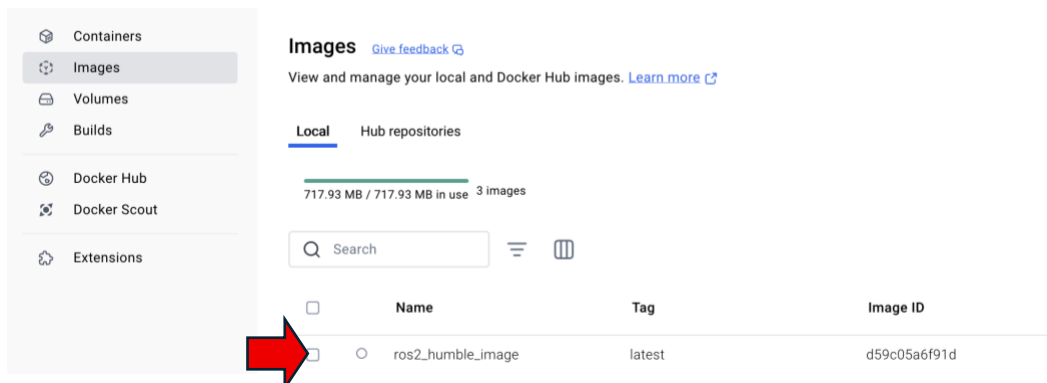
The commands in the *Dockerfile* are run to build a new image which we'll use in this class.

3. In the Docker terminal, navigate to your *ENGR857* repository and build a new Docker image:

```
docker image build -t ros2_humble_image .
```

- a. The `-t` flag (“tag”) assigns a human-readable name to the new image (e.g., `ros2_humble_image`).
- b. The `.` (period) tells Docker to look for the *Dockerfile* in the current directory.

4. You should see the new image in the Docker Desktop application, under “Images”.



3.4 Run a container with a mounted volume

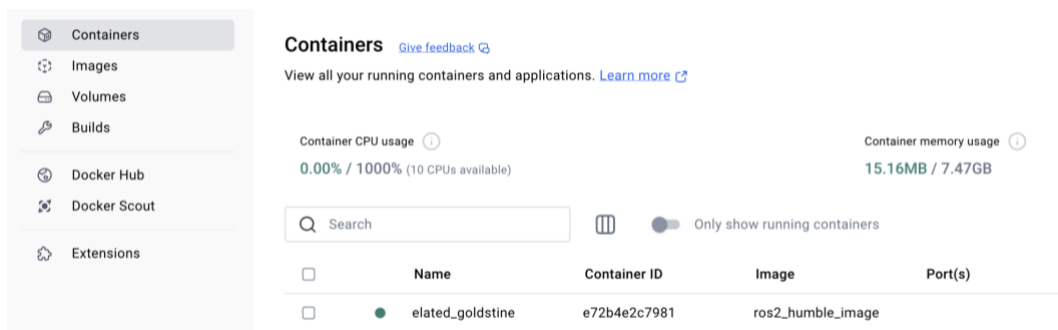
1. In your *ENGR857* repository, create a new directory called *ros2_ws*. Any modifications made in the Docker container are not preserved, so we will mount this new directory as a *volume* in the container to enable persistent changes.
2. Start the container with an interactive shell and mounted volume. Note the syntax of the command is as follows:

```
docker run -it -v VOLUME_PATH IMAGE_NAME
```

For instance, my laptop has an Apple Silicone chip, so I would run:

```
docker run -it -v ~/ENGR857/ros2_ws:/ros2_ws ros2_humble_image
```

- a. The `-it` flag turns on “interactive TTY”, allowing us to enter commands directly into the container’s terminal.
 - b. The `-v` flag specifies the volume to be mounted inside the container. In this case, it binds my local directory `~/ENGR857/ros2_ws` to the directory `/ros2_ws` within the container. Any changes made in `/ros2_ws` in the container will then also be saved to my local machine and transferable to another Docker session.
 - c. `ros2_humble_image` is the name of the image to run.
3. Confirm the container was created successfully. Your command line should now indicate that you are logged in as “root” on a machine name comprised of letters and numbers, and you should see it listed under “Containers”.



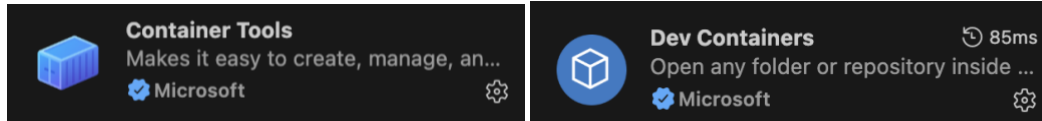
4. The container can now be treated as a Unix environment with ROS 2 installed. For example, if you run the `ls` command, you can view all the files in the `/root` directory.

```
akubota@sfs-wifi-dhcp-10-143-186-21 ENGR857 % docker run -it -v ~/Documents/ENGR857/ros2_ws:/ros2_ws ros2_humble_image
root@e72b4e2c7981:/# ls
bin boot dev etc home lib log media mnt opt proc root ros2_ws ros_entrypoint.sh run sbin srv sys tmp
```

5. Note that the container will continue running if the shell session is active (exiting the shell will not automatically stop the container). You can simultaneously stop and exit the container pressing `Ctrl+D`.

3.5 Connect Docker to VS Code

1. Back under the Extensions , search for “Containers” and install both “Container Tools” and “Dev Containers”.



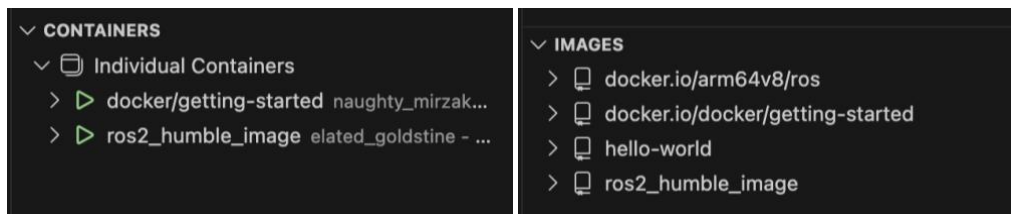
2. If you are using a Mac (not sure if this is required on Windows), make sure the Docker executable is on your PATH. Add the following lines to your ~/.zprofile:

```
# Add Docker Desktop for Mac (docker)
export PATH="$PATH:/Applications/Docker.app/Contents/Resources/bin/"
```

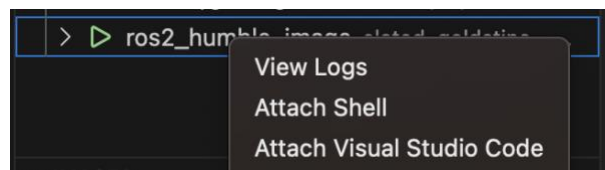
3. Exit VS Code and reopen it to make sure the changes take effect.

4. You should now see a Containers  icon on the left side bar. Click it.

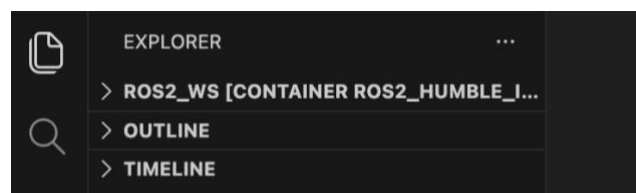
5. Your Docker containers and images should now be listed.



6. Right click on your *ros2_humble_image* container, and select “Attach Visual Studio Code”. Alternatively, you can select “Attach Shell” to open a terminal connected to this container.



7. A new VS Code window should open. You can open the /ros2_ws/ directory you mounted to this container, though it is still empty.



3.6 Create a new ROS 2 package

1. In the container, create a `src` directory under your `ros2_ws` folder to follow the typical structure of a ROS 2 workspace, as shown.

```
ROS_workspace_folder/  
  src/  
    cpp_package_1/  
      CMakeLists.txt  
      include/cpp_package_1/  
      package.xml  
      src/  
  
    py_package_1/  
      package.xml  
      resource/py_package_1  
      setup.cfg  
      setup.py  
      py_package_1/  
    ...  
    cpp_package_n/  
      CMakeLists.txt  
      include/cpp_package_n/  
      package.xml  
      src/
```

2. Build the workspace by running the following command in the `ros2_ws` directory:

```
colcon build
```

3. You'll notice that three folders were generated:
 - a. `./build/` contains immediate build artifacts
 - b. `./install/` holds the compiled packages and setup files
 - c. `./log/` stores detailed build logs

3.7 Create and compile custom ROS 2 nodes

1. Now we're (finally) on to the fun part! Complete the "Writing a simple publisher and subscriber" ROS 2 tutorial in the language of your choice. (It might be worth looking through both pages before starting to compare the similarities and differences between the code, steps to build and run, etc.)

For Python: <https://docs.ros.org/en/humble/Tutorials/Beginner-Client-Libraries/Writing-A-Simple-Py-Publisher-And-Subscriber.html>

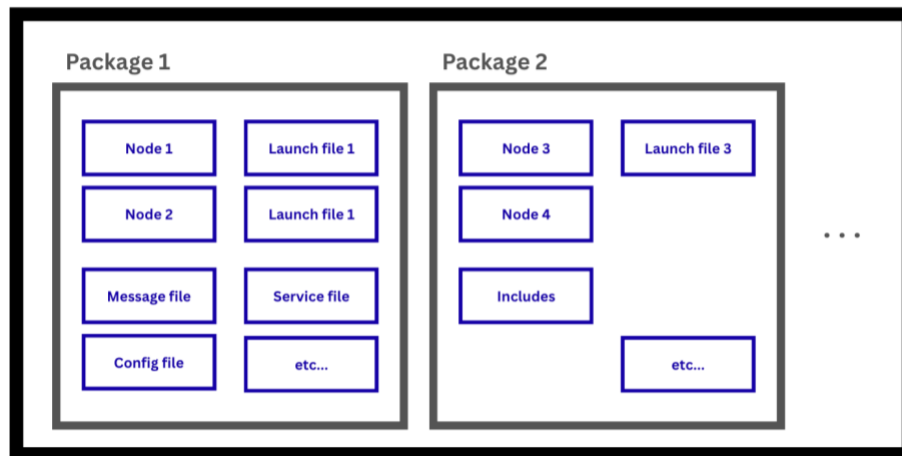
For C++: <https://docs.ros.org/en/humble/Tutorials/Beginner-Client-Libraries/Writing-A-Simple-Cpp-Publisher-And-Subscriber.html>

3.8 Analyze your workspace

Congratulations, now you can send and receive data over a *topic*! Topics are one of the main ways we can communicate between *nodes*.

1. Note that ROS uses the following terminology (see the figure below for an example visual representation of some of these concepts):
 - a. A *workspace* (e.g., *ros2_ws*) is a directory that houses ROS 2 packages, typically associated with one project.
 - b. A *package* (e.g., *py_pubsub*, *cpp_pubsub*) can be thought of as a library. The package may contain relevant files, such as code, executables, non-ROS libraries, datasets, configuration files, etc.
 - c. A *node* (e.g., “minimal_publisher”, “minimal_subscriber”) is an individual process.
 - d. A *topic* is the channel through which nodes publish and subscribe to messages.
 - e. A *launch* file can be used to execute multiple nodes at once.
 - f. A *message* file lets you define custom message structures to send over a topic.
 - g. The *underlay* is the base set of ROS 2 packages you can call from any workspace.
 - h. Your *overlay* are the packages defined in your workspace. You can switch between overlays by sourcing the *install/setup.bash* file from the desired workspace.

Workspace (overlay)



2. With the above figure as reference, **draw a diagram of your *ros2_ws***, making sure to label:
 - a. Your workspace
 - b. Your package
 - c. Any nodes
3. **Draw a diagram demonstrating how messages are sent between your nodes.** You may find it helpful to reference the [ROS 2 topics tutorial](#). Make sure to label:
 - a. Any nodes
 - b. Any topics
 - c. Use directed edges (i.e., arrows) to show how messages are sent/received

3.9 Write new nodes

Last stretch! Now, let's practice writing our own nodes.

1. Write a node called “num_talker” that:
 - a. Generates random integers
 - b. Publishes these integers to a topic named */numbers* (The type of this topic should **not** be *std_msgs/String*)
 - c. Prints these integers to the console
2. Write a second node called “square” that:
 - a. Subscribes to the topic */numbers*
 - b. Squares the value it receives
 - c. Prints the squared value out to the console
3. Test and debug your code. Note that this can be difficult for ROS 2 and you may find the following commands helpful:

```
ros2 topic list           # Check all active topics
ros2 topic info /numbers # Verify topic type and connections
ros2 topic echo /numbers # Inspect message content
```

Some common issues:

- a. Topic names must match exactly (underscores, cases, etc.)
- b. Make sure your *package.xml* and *setup.py* files are correct, including all dependencies and entry points

You can also try Python's debugger to step through your code (see the [docs](#) for more info).

4. **Record a video of your nodes running** and upload it to Canvas. Requirements:
 - a. Values printed by “num_talker” and “square” are both visible
 - b. Video is ~10 seconds long
 - c. Video includes a brief explanation (written or spoken) of how your code works.
5. Thoroughly comment your code, making sure to include:
 - a. A **header** at the top of each file you wrote (*num_talker*, *square*). See [examples](#) here. Make sure to include:
 - i. Your name
 - ii. The filename
 - iii. A brief description of the file
 - b. A documentation string (“**docstring**”) for each method you wrote. Docstrings provide a description of the associated class, function, or method. Here are some examples for [Python](#) and [C++](#). Make sure to include:

- i. A brief description of the method
 - ii. A description of each parameter it takes (and their expected types in Python)
 - iii. The return value (and its expected type in Python)
 - c. **Comments** for any line of code that is not self-explanatory
6. Commit and push your code to GitHub. Great work, you're done!